

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – Comparative Analysis

COURSE NAME: Data Structures

COURSE CODE:CSA03

COURSE OUTCOME COVERED

CO: Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data **(BL3)**

Assessment Tool Weightage: 30%

QUESTIONS: EACH QUESTION CARRIES: 10 MARKS DURATION : 1 HOUR

S.No	Questions
-------------	------------------

- | | |
|----|---|
| 1. | A system needs to store large data and perform frequent search operations. For this, compare Hashing and Sorting-based search (Binary Search) in terms of: <ul style="list-style-type: none">a) Time complexityb) Space usagec) Update operationsd) Real-world applicability |
|----|---|

Conclude which is better for dynamic systems.

- | | |
|----|---|
| 2. | Compare Quick Sort and Merge Sort based on: <ul style="list-style-type: none">a) Best, average, and worst-case complexityb) Stabilityc) Memory usaged) Practical performance |
|----|---|

Explain why Quick Sort is often preferred in real applications despite its worst-case complexity.

- | | |
|----|---|
| 3. | A system needs to repeatedly: <ul style="list-style-type: none">a) Insert elementsb) Retrieve the highest priority element |
|----|---|

Compare Heap and Sorted Array in terms of:

- a) Insertion time
- b) Deletion time
- c) Efficiency in real-time systems

Suggest which structure is better for scheduling systems.

- | | |
|----|---|
| 4. | Compare Linear Probing and Separate Chaining for collision handling in hashing based on: <ul style="list-style-type: none">a) Clustering effectb) Memory usage |
|----|---|

- c) Performance under high load factor

Which method is more suitable for large-scale applications and why?

5. Compare Sequential Sorting and Parallel Sorting (Multi-threading / MapReduce) in terms of:
 - a) Execution time
 - b) Scalability
 - c) Overhead
 - d) Suitability for small vs large datasets

Explain why parallel sorting is not always the best choice.

6. Compare separate chaining versus open addressing (linear probing) for a hash table storing 10 million key-value pairs with a load factor of 0.75. Analyze memory usage, cache performance, worst-case lookup time, and deletion complexity. Under what scenarios would you choose one over the other?
7. Compare linear probing, quadratic probing, and double hashing as collision resolution strategies. Analyze primary clustering, secondary clustering, and average probe counts for successful and unsuccessful searches. Which method offers the best cache performance and why?
8. Compare a binary heap (priority queue) versus a balanced binary search tree (e.g., AVL or Red-Black) for implementing a scheduler that requires both min extraction and arbitrary key updates. Analyze time complexity for insert, extract-min, and decrease-key operations. Which data structure is more memory-efficient?
9. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays?
10. Compare heap sort versus quick sort for sorting 1 billion integers in a real-time system with strict latency requirements (no operation can exceed 1 second). Analyze worst-case time complexity, space complexity, and predictability. Which algorithm guarantees $O(n \log n)$ worst-case and why would you choose it despite quick sort's faster average case?
11. Compare radix sort (LSD, base 256) versus comparison-based sorts (quick sort, merge sort) for sorting 100 million 32-bit integers. Analyze time complexity in terms of $O(d \times n)$ vs. $O(n \log n)$, memory usage (counting array), and practical performance on modern CPUs. When does radix sort lose its advantage?
12. Compare insertion sort versus quick sort for sorting small arrays (size < 50). Analyze constant factors, branch prediction, cache locality, and overhead of recursive calls. Why do production sorting implementations (e.g., Timsort, Introsort) switch to insertion sort for small subarrays?
13. Compare single-threaded quick sort, parallel quick sort with 8 threads, and parallel merge sort with 8 threads on a shared-memory multi-core system. Analyze theoretical speedup (Amdahl's law), overhead of synchronization, load imbalance, and memory bandwidth contention. Which algorithm scales better and why?

14. Compare MapReduce sorting (total order partitioning) versus parallel merge sort on a shared-memory machine for sorting 10 TB of data distributed across 100 nodes. Analyze network communication cost, fault tolerance, disk I/O, and scalability. Under what conditions does MapReduce become necessary despite its higher overhead?
15. Compare GPU bitonic sort (CUDA) versus CPU parallel quick sort (64 cores) for sorting 1 billion integers. Analyze parallel time complexity ($O(\log^2 n)$ vs. $O((n \log n)/p)$), memory hierarchy (global vs. shared memory), warp divergence, and PCIe transfer overhead. For what input sizes does GPU sorting become advantageous?
16. Compare linear probing and cuckoo hashing for a read-heavy workload (90% lookups, 10% inserts) with 100 million keys. Analyze worst-case lookup time guarantees, insertion failures, rehashing costs, and memory utilization. Which hash table variant offers better predictable latency?
17. Compare binary heaps and Fibonacci heaps for implementing Dijkstra's shortest path algorithm on a dense graph with 100,000 nodes and 10 million edges. Analyze amortized time complexity for decrease-key operations, memory overhead, and practical performance. Why is Fibonacci heap rarely used in practice despite better theoretical bounds?
18. Compare Timsort (adaptive hybrid sort) versus traditional merge sort for sorting real-world data with the following characteristics: (a) already sorted array, (b) reverse sorted array, (c) array with many duplicate keys, (d) array with random order. Analyze best-case, average-case, and worst-case time complexity for each scenario.
19. Compare hash table with separate chaining versus binary search tree for implementing a dictionary in a multi-threaded environment. Analyze locking granularity, concurrent access patterns, resizing complexity, and cache behavior. Which structure supports more scalable concurrent operations?
20. Compare counting sort, radix sort, and bucket sort for sorting floating-point numbers in the range $[0.0, 1.0)$ with uniform distribution. Analyze how each algorithm handles non-integer keys, distribution sensitivity, memory usage, and stability. Which algorithm is most suitable for sorting 100 million uniformly distributed floats and why?
21. Compare counting sort, radix sort, and bucket sort for sorting floating-point numbers in the range $[0.0, 1.0)$ with uniform distribution. Analyze how each algorithm handles non-integer keys, distribution sensitivity, memory usage, and stability. Which algorithm is most suitable for sorting 100 million uniformly distributed floats and why?
22. Compare counting sort, radix sort, and bucket sort for sorting floating-point numbers in the range $[0.0, 1.0)$ with uniform distribution. Analyze how each algorithm handles non-integer keys, distribution sensitivity, memory usage, and stability. Which algorithm is most suitable for sorting 100 million uniformly distributed floats and why?
23. Compare Quadratic Probing and Double Hashing in terms of:

- a) Clustering issues
 - b) Collision resolution efficiency
 - c) Suitability for dense hash tables
- 24.** Compare Best-case and Worst-case time complexity of sorting algorithms based on the following:
- a) Practical significance
 - b) Impact on algorithm selection
- 25.** Compare Recursive and Iterative implementations of Quick Sort in terms of:
- a) Space complexity (stack usage)
 - b) Ease of implementation
 - c) Risk of stack overflow
- 26.** Compare Merge Sort and Radix Sort based on the following:
- a) Type of sorting technique
 - b) Input limitations
 - c) Performance on large datasets
- 27.** Compare two methods of building a heap:
- a) Bottom-up heap construction
 - b) Repeated insertion into heap
- Which is more efficient and why?
- 28.** Compare Heap-based priority queue and array-based priority queue in terms of:
- a) Insert operation
 - b) Delete operation
 - c) Overall efficiency
- 29.** Compare Binary Heap and Binary Search Tree (BST) based on the following:
- a) Searching capability
 - b) Insert/delete operations
 - c) Typical applications
- 30.** Compare Internal sorting and External sorting based on the following:
- a) Memory usage
 - b) Data size handled
 - c) Performance considerations
- 31.**
- a) Compare Parallel Merge Sort and Sequential Merge Sort in terms of:
 - b) Execution time
 - c) Overhead due to parallelism
 - d) Suitability for large datasets

32. Compare Single-core (sequential) sorting and multi-core sorting based on the following:
 - a) Performance improvement
 - b) Synchronization overhead
 - c) Practical challenges
33. Compare Time complexity and Space complexity in terms of:
 - a) Trade-offs
 - b) Situations where one is prioritized over the other
34. Compare Direct Addressing and Hashing based on the following:
 - a) Memory requirements
 - b) Flexibility
 - c) Real-world applicability
35. Compare the effect of load factor in hash tables with input size in sorting algorithms based on the following:
 - a) Performance degradation
 - b) Optimization strategies
36. Compare Heap Sort and Quick Sort in terms of:
 - a) Worst-case guarantees
 - b) Practical performance
 - c) Use cases
37. Compare Radix Sort and Counting Sort based on the following:
 - a) Working mechanism
 - b) Range of input values
 - c) Efficiency
38. Compare Single-threaded sorting and distributed sorting (e.g., cluster-based) based on the following:
 - a) Scalability
 - b) Resource requirements
 - c) Complexity of implementation
39. Compare Binary Heap and Fibonacci Heap in terms of:
 - a) Insert operation
 - b) Decrease-key operation
 - c) Practical usability in real systems
40. Compare two approaches for searching based on the following:
 - a) Sorting data first and then applying binary search

- b) Using hashing for direct lookup
Which is better for static data? Which is better for dynamic data?

Code of Conduct:

I G VENKATA GANESH - 192425381 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G V GANESH

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

7) Comparative Analysis: Linear Probing vs Quadratic Probing vs Double Hashing

1) Comparison Table

Feature	Linear Probing	Quadratic Probing	Double Hashing
Probe sequence			
Primary clustering	High	No	No
Secondary clustering	Yes	Yes	Very low / No significant
Probe pattern	Consecutive locations	Increasing jumps	Key-dependent jumps
Successful search	Good at low load, degrades quickly	Better than linear	Generally best
Unsuccessful search	High probes at high load	Moderate	Generally lowest
Cache performance	Excellent	Good	Poor
Implementation	Very simple	Simple	More complex
Memory locality	Excellent	Moderate	Poor
Performance at high load factor	Poor	Better	Best

2) Linear Probing

In linear probing, when a collision occurs, the next consecutive location is checked.

Example:

$$h(k) = 5$$

$$5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9$$

Primary Clustering

Linear probing suffers heavily from primary clustering.

When several keys collide, they form a continuous block:

[5][6][7][8][9][10]

↑ cluster

New keys that hash to any position inside or near this cluster are likely to join the same cluster.

Secondary Clustering

Linear probing also has secondary clustering in the sense that keys with the same initial hash value follow the same probe sequence.

Probe Counts

For load factor , standard approximations are:

Successful search:

$$E_s \approx \frac{1}{\alpha} \ln(1 - \alpha)$$

Unsuccessful search:

$$E_u \approx \frac{1}{1 - \alpha}$$

Therefore, unsuccessful searches become expensive when the table becomes nearly full.

3) Quadratic Probing

Quadratic probing uses squared increments:

$$h(k), h(k)+1^2, h(k)+2^2, h(k)+3^2, \dots$$

For example, if the original position is 5:

$$5 \rightarrow 6 \rightarrow 9 \rightarrow 14 \rightarrow \dots$$

Primary Clustering

Quadratic probing eliminates primary clustering because keys do not simply form long consecutive blocks.

Secondary Clustering

However, it still suffers from secondary clustering.

If two keys have the same initial hash value, they follow the same quadratic probe sequence.

Probe Counts

Quadratic probing generally requires fewer probes than linear probing, particularly when the table has moderate or high occupancy.

Its exact average probe count depends on the table size, probing parameters, and load factor, so unlike linear probing and ideal double hashing, there is not one universally applicable simple formula.

4) Double Hashing

Double hashing uses a second hash function to determine the size of the jump:

For example:

$$h_1(k) = 5$$

$$h_2(k) = 3$$

$$5 \rightarrow 8 \rightarrow 11 \rightarrow 14 \rightarrow \dots$$

Different keys usually have different probe sequences.

Primary Clustering

Double hashing eliminates primary clustering.

Keys that collide at the same initial position can use different step sizes.

Secondary Clustering

It also largely eliminates secondary clustering because different keys can have different probe sequences even when their initial positions are the same.

Probe Counts

Under the usual uniform-hashing approximation:

Successful search:

Unsuccessful search:

Thus, double hashing generally gives the best average probe performance among these three methods.

5) Example of Average Probe Performance

Consider a table with a **load factor of 0.75**.

Approximate values:

Method	Successful Search	Unsuccessful Search
Linear Probing	≈ 2.50 probes	≈ 8.50 probes
Double Hashing	≈ 1.85 probes	≈ 4.00 probes
Quadratic Probing	Usually between linear and double hashing	Usually between linear and double hashing

This shows that as the table becomes more crowded, **linear probing suffers significantly because of clustering**.

6) Which Method Has the Best Cache Performance?

Linear probing has the best cache performance.

The main reason is spatial locality.

Linear probing accesses consecutive memory locations:

Index $10 \rightarrow 11 \rightarrow 12 \rightarrow 13 \rightarrow 14$

Since these locations are close together, they are likely to be loaded into the CPU cache together.

Quadratic Probing

Quadratic probing accesses locations with increasing jumps:

$10 \rightarrow 11 \rightarrow 14 \rightarrow 19 \rightarrow \dots$

Therefore, its cache locality is lower than linear probing.

Double Hashing

Double hashing can jump to widely separated locations:

$10 \rightarrow 37 \rightarrow 64 \rightarrow 91 \rightarrow \dots$

Therefore, it has the worst cache locality of the three.